Binary and digital logic 2

MEGN 441
Zhaoda Du

COLORADO SCHOOL OF MINES

Administrative stuff

- Lab 2 continues next week, 2/5
- Homework 3 posted today
  - Due 09/17

Motivation for the day

- We want to measure the acceleration of our robot, so we get an Inertial Measurement Unit (IMU)
- The BNO055 9-axis IMU has an accelerometer, gyroscope and magnetometer
- However, we want to improve the resolution of the IMU

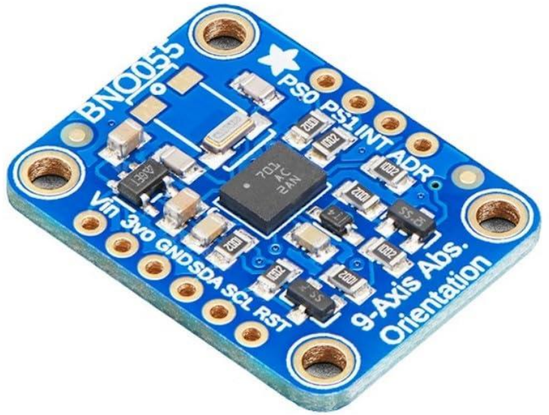
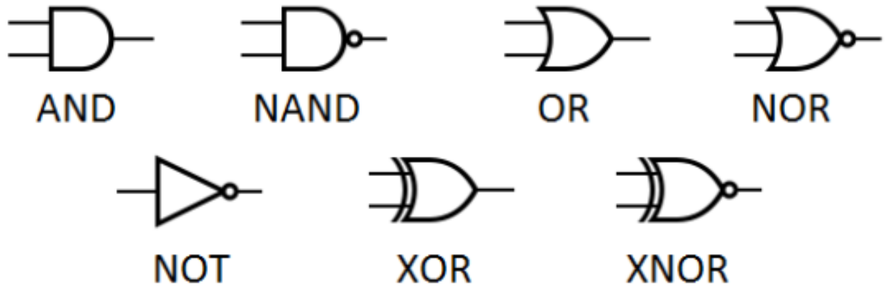


Table 3-7: Default sensor configuration at power-on

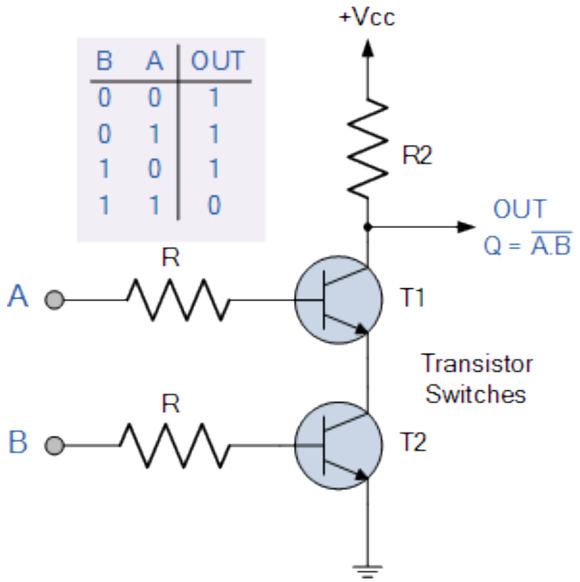
Sensors	Parameters	Value
Accelerometer	Power Mode	NORMAL
	Range	+/- 4g
	Bandwidth	62.5Hz
	Resolution	14 bits
Gyroscope	Power Mode	NORMAL
	Range	2000 °/s
	Bandwidth	32Hz
	Resolution	16 bits
Magnetometer	Power Mode	FORCED
	ODR	20Hz
	XY Repetition	15
	Z Repetition	16
	Resolution x/y/z	13/13/15 bits

Digital logic



https://learn.adafruit.com/binary-boolean-and-logic/logic-gates

Physical NAND Gate



Truth Tables

A	B	AND	NAND	OR	NOR	XOR	XNOR
0	0	0	1	0	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	0	1	0	0	1

https://www.electronics-tutorials.ws/logic/logic_5.html

Bitwise Logical Operators

- Bitwise Operators
  - & : AND
  - | : OR
  - ^ : XOR
  - ~ : NOT

A: 0000 1111

B: 0101 0101

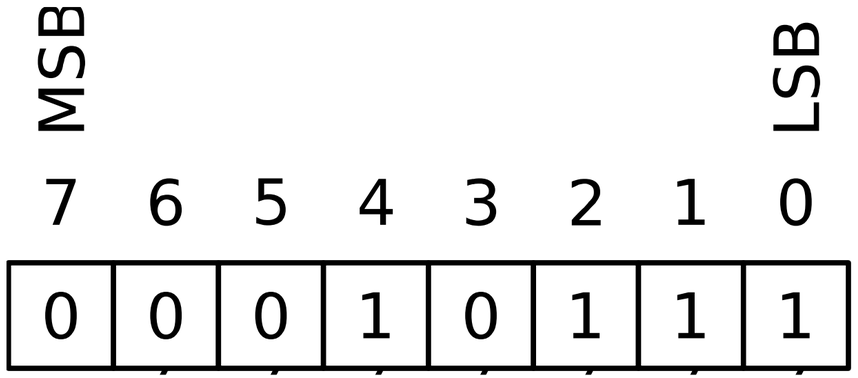
~B:

A & B

A | B

A ^ B

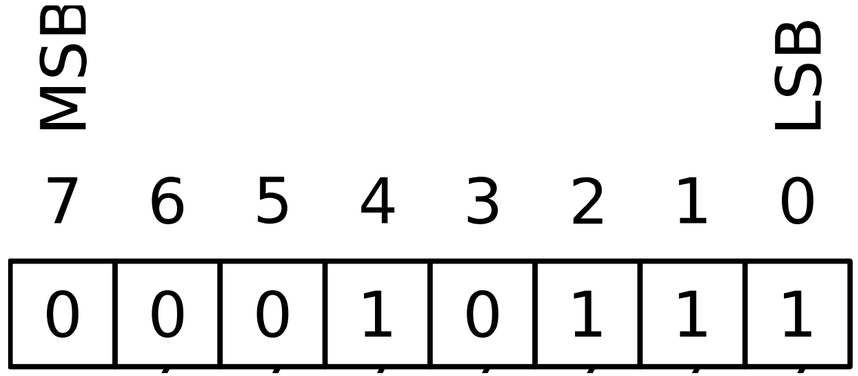
Anatomy of a byte



- MSB – Most Significant Bit
- LSB – Least Significant Bit

- Can represent a number, but can also represent data in other forms

Big Endian vs Little Endian

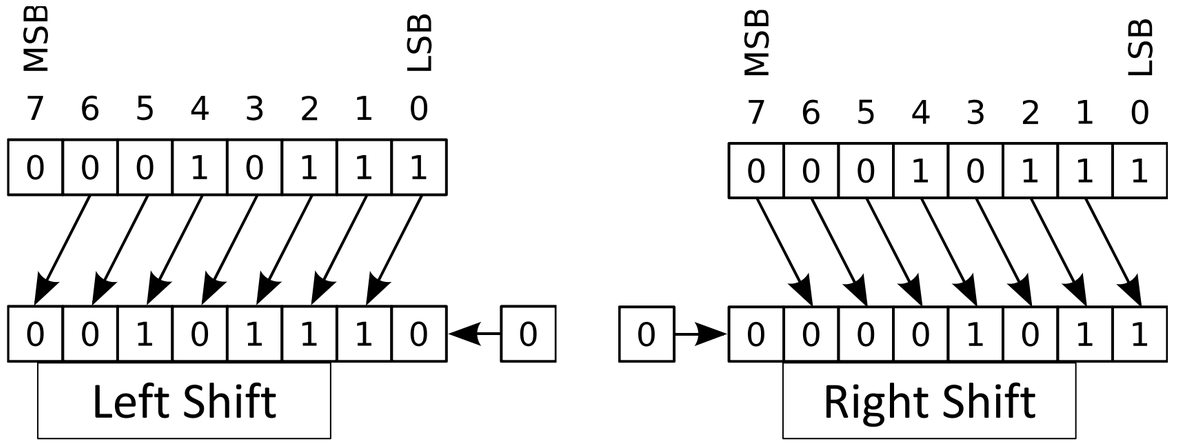


- Indicates the order of data
  - Big Endian = MSB first
  - Little Endian = LSB first
- Can refer to bytes or bits, typically bit order is big endian
- Data storage (ATmega328p(microchip on Arduino) is little endian, least significant byte is stored in the lowest number address)
- Data transmission is often little endian, least significant byte is sent first
  - NOTE: you can still send the least significant **bytes** first while sending those bytes with the most significant **bit** first

Registers

- **Registers are very fast memory bits** located inside the CPU. They are used to temporarily hold data for calculations or control.
- Arduino UNO has 32 8-bit register in CPU (called general purpose registers)
  - (see ATmega328p datasheet pg 8)
- Multiple other registers throughout, used for various purposes
  - (see ATmega328p datasheet pg 5)
- Some are connected to pins you can access
- Full list of registers, ATmega328p datasheet pg 423-6

Bit shifting



- Bit Shifting
  - Left Shift: <<
    - x << n
    - The bits of number “x” are shifted “n” bits to the left (left-most n bits are lost)
  - Right Shift: >>
    - x >> n
    - The bits of number “x” are shifted “n” bits to the right (right-most n bits are lost)

Bit shifting is used for efficient math, manipulating individual bits, creating masks, packing/unpacking data, and controlling hardware at the binary level.

General Rule

- **x << n** → equivalent to **x × 2^n**
- **x >> n** → equivalent to **x ÷ 2^n**

This works perfectly for **unsigned integers**.

For **signed integers**, right shift can behave differently:

Bitmasks

- Used to set certain bit values WITHOUT knowing the original information
- Convert 1s -> 0s, use 0s with an AND, leave the rest of the bits as 1s
- Convert 0s -> 1s, use 1s with an OR, leave the rest as 0s
- Examples, b = 1011 0100

Change bits 3:2 to 1

mask = 0000 1100

could do

mask = (1<<3) | (1<<2)

b | mask = 1011 1100

b |= mask

Change bits 7,5 to 0

mask = 0101 1111

could do

mask = ~((1<<5)|(1<<7))

b & mask = 0001 0100

b = 1100 0010
Change bits 1:0 to 1

0010 1011

b = 0110 1111
Change bits 6,2 to 0

1100 0011

Port registers

- Ports are registers inside the microcontroller connected to pins
  - Allow for external reading and writing of certain registers
- ATmega328p ports are associated with 3 registers
  - **Data Register** (e.g. PORTB): storage of port bit values
    - If pin is INPUT the Data Register controls the Pullup resistor
  - **Data Direction Register** (e.g. DDRB): dictates whether pin is INPUT (0) or OUTPUT (1)
  - **Port Input Pin** (e.g. PINB): stores the current value of the pin

  - (see ATmega328p datasheet pg 75-6)

Port registers

- PORTx and DDRx registers can directly be written to!
  - PINx is read-only
- DDRB = 0b00100000 (pin 5 is output, rest are input)
  - In Arduino, the register names can be used to edit the register directly
- PORTB = 0b00100100 (set pin 5 to high, pin 2 to pull-up input)

- Use bit-masking to avoid accidentally changing other pins!

```
DDRB |= (1 << 5);
PORTB |= (1 << 5);
```

Example:

pinMode(3, INPUT_PULLUP) using masks and port registers?

mask = 0b00000 1000

mask = 1<<3

set PORTB bit 3 to 1

PORTB = PORTB | mask

set DDRB bit 3 to 0

mask2 = ~mask

= 1111 0111

DDRB = DDRB & mask2

Motivation for the day

- We want to measure the acceleration of our robot, so we get an Inertial Measurement Unit (IMU)
- The BNO055 9-axis IMU has an accelerometer, gyroscope and magnetometer
- However, we want to improve the resolution of the IMU

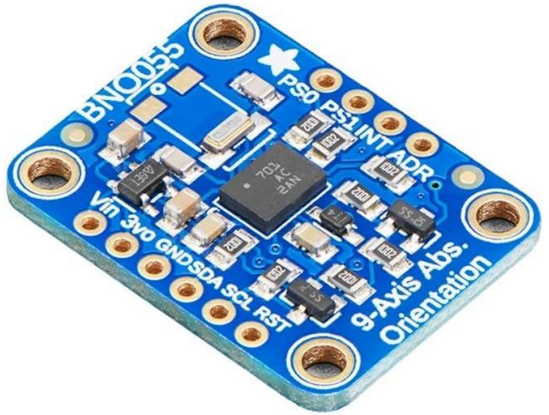


Table 3-7: Default sensor configuration at power-on

Sensors	Parameters	Value
Accelerometer	Power Mode	NORMAL
	Range	+/- 4g
	Bandwidth	62.5Hz
	Resolution	14 bits
Gyroscope	Power Mode	NORMAL
	Range	2000 °/s
	Bandwidth	32Hz
	Resolution	16 bits
Magnetometer	Power Mode	FORCED
	ODR	20Hz
	XY Repetition	15
	Z Repetition	16
	Resolution x/y/z	13/13/15 bits

Port registers

- BNO055 IMU Configuration Example (BNO055_read_write_reg)
  - Accelerometer
    - Default: +/-4g, bits 1:0 = 0b01
    - Desired Value: +/-2g, bits 1:0 = 0b00
  - Gyroscope
    - Default: 2000 deg/sec, bits 2:0 = 0b000
    - Desired Value: 250 deg/sec, bits 2:0 = 0b011

Possible configurations

Table 3-8: Accelerometer configurations

Parameter	Values	[Reg Addr]: Reg Value
G Range	2G	[ACC_Config]: xxxxx00b
	4G	[ACC_Config]: xxxxx01b
	8G	[ACC_Config]: xxxxx10b
	16G	[ACC_Config]: xxxxx11b
Bandwidth	7.81Hz	[ACC_Config]: xxx000xxb
	15.63Hz	[ACC_Config]: xxx001xxb
	31.25Hz	[ACC_Config]: xxx010xxb
	62.5Hz	[ACC_Config]: xxx011xxb
	125Hz	[ACC_Config]: xxx100xxb
	250Hz	[ACC_Config]: xxx101xxb
	500Hz	[ACC_Config]: xxx110xxb
	1000Hz	[ACC_Config]: xxx111xxb
Operation Mode	Normal	[ACC_Config]: 000xxxxxb
	Suspend	[ACC_Config]: 001xxxxxb
	Low Power 1	[ACC_Config]: 010xxxxxb
	Standby	[ACC_Config]: 011xxxxxb
	Low Power 2	[ACC_Config]: 100xxxxxb
	Deep Suspend	[ACC_Config]: 101xxxxxb

Table 3-9: Gyroscope configurations

Parameter	Values	[Reg Addr]: Register value
Range	2000 dps	[GYR_Config_0]: xxxx000b
	1000 dps	[GYR_Config_0]: xxxx001b
	500dps	[GYR_Config_0]: xxxx010b
	250 dps	[GYR_Config_0]: xxxx011b
	125 dps	[GYR_Config_0]: xxxx100b
Bandwidth	523Hz	[GYR_Config_0]: xx000xxxb
	230Hz	[GYR_Config_0]: xx001xxxb
	116Hz	[GYR_Config_0]: xx010xxxb
	47Hz	[GYR_Config_0]: xx011xxxb
	23Hz	[GYR_Config_0]: xx100xxxb
	12Hz	[GYR_Config_0]: xx101xxxb
	64Hz	[GYR_Config_0]: xx110xxxb
	32Hz	[GYR_Config_0]: xx111xxxb
Operation Mode	Normal	[GYR_Config_1]: xxxx000b
	Fast Power up	[GYR_Config_1]: xxxx001b
	Deep Suspend	[GYR_Config_1]: xxxx010b
	Suspend	[GYR_Config_1]: xxxx011b
	Advanced Powersave	[GYR_Config_1]: xxxx100b

Port registers

- BNO055 IMU Configuration Example (BNO055_read_write_reg)
  - Accelerometer
    - Default: +/-4g, bits 1:0 = 0b01
    - Desired Value: +/-2g, bits 1:0 = 0b00
  - Gyroscope
    - Default: 2000 deg/sec, bits 2:0 = 0b000
    - Desired Value: 250 deg/sec, bits 2:0 = 0b011

Port registers

- BNO055 IMU Configuration Example (BNO055_read_write_reg)
  - Configuration is controlled by the “ACC_Config” and “GYRO_Config_0” registers
    - BNO055 datasheet pages 77-8 show the register addresses and bit values
  - NOTE: also need to set the PAGE_ID register to 1
    - These configuration registers can only be accessed after changing to page 1 (BNO055 datasheet page 50)